# StaffSync HRMS  Comprehensive Project Documentation

## 1. System Overview
**StaffSync** is an enterprise Human Resource Management System engineered to streamline workforce operations, time and attendance, multi-tiered leave approvals, employee learning, organizational announcements, and compliance auditing.

The system is architected as a modular full-stack application featuring a React 19 single-page application frontend and an Express REST API backend, supported by a dual-engine storage architecture (MongoDB Atlas with automatic in-memory fallback).

---

## 2. Technology Stack

| Layer | Technologies & Libraries |
| :--- | :--- |
| **Frontend Framework** | React 19 (‘react‘, ‘react-dom‘), TypeScript 5.8+ |
| **Styling & Design System** | Tailwind CSS v4 (‘@tailwindcss/vite‘), Lucide Icons (‘lucide-react‘) |
| **Build & Tooling** | Vite 6, ‘tsx‘ Node TypeScript runtime, ESLint |
| **Backend Runtime** | Node.js, Express 4.x, HTTP/REST |
| **Database & ORM** | MongoDB Atlas, Mongoose 8.x, In-Memory MemoryStore Fallback |
| **Security & Auth** | JSON Web Tokens (‘jsonwebtoken‘), ‘bcryptjs‘, CORS middleware |
| **Charts & Data Viz** | Recharts 2.x, Custom SVG Data Metric Cards |
| **Data Export** | Structured CSV Generator (UTF-8 formatted) |

---

## 3. Project Directory Structure

StaffSync is organized into distinct layers for frontend client assets, server business logic, and modular packages:

```
/
    staffsync/                      # Modular Project Root
        client/                     # Frontend Application (Client)
            api/                    # API HTTP client & service wrappers
                client.ts           # Axios/Fetch client with JWT interceptor
            components/             # Reusable UI components
```

```
                Header.tsx             # Workspace navigation, notifications & theme
  toggle
                Sidebar.tsx            # Role-based sidebar navigation
                ui/                    # Atomic design components (Badge, Modal, Stat
  Card)
            context/                   # Global React state providers
                AuthContext.tsx        # Authentication & persona state
                ThemeContext.tsx       # Dark / Light theme switcher
                ToastContext.tsx       # In-app notifications & alerts
            types/                     # TypeScript shared interfaces
                index.ts               # Domain type declarations
            views/                     # Feature screen views
                LoginView.tsx          # Credentials & 1-click persona login
                DashboardView.tsx      # Role-tailored metrics & widgets
                EmployeesView.tsx      # Workforce directory & profile viewer
                AttendanceView.tsx     # Punch clock, timesheets & corrections
                LeavesView.tsx         # Leave applications & approval workflows
                DepartmentsView.tsx    # Business unit budgeting & hierarchy
                TrainingsView.tsx      # Course enrollments & progress tracking
                AnnouncementsView.tsx  # Company-wide notices & alerts
                ReportsView.tsx        # Analytical charts & CSV exports
                AuditLogsView.tsx      # Immutable regulatory audit trails
                ProfileView.tsx        # Personal profile & password security
            App.tsx                    # Root routing & layout controller
            index.css                  # Tailwind CSS v4 theme variables
            main.tsx                   # React DOM entrypoint

        server/                        # Backend Application (Server)
            routes/                    # REST API Route Handlers
                authRoutes.ts          # Login, token validation, demo personas
                employeeRoutes.ts      # Employee CRUD & manager lookups
                departmentRoutes.ts    # Department management
                attendanceRoutes.ts    # Clock-in, clock-out, manual adjustments
                leaveRoutes.ts         # Leave balance, requests & two-tier decisions
                announcementRoutes.ts  # Announcements publishing
                trainingRoutes.ts      # Workshop creations & progress updates
                notificationRoutes.ts  # In-app alert read/unread polling
                reportRoutes.ts        # Analytics aggregates & CSV downloads
                auditLogRoutes.ts      # Audit event querying
            auth.ts                    # JWT token generation & auth middleware
            db.ts                      # Hybrid Mongoose / in-memory database engine
            server.ts                  # Express server entry point & API mount
```

```
    tests/                              # Quality Assurance Test Suite
        api.test.ts                     # 17-point automated verification tests
    PHASE_WISE_DOCUMENTATION.md         # Detailed phase-by-phase implementation logs
    PROJECT_DOCUMENTATION.md            # This comprehensive architecture reference
    STAFF_SYNC.md                       # Project identity, quick-start & module index
    package.json                        # Dependencies & build scripts
    tsconfig.json                       # TypeScript strict compiler options
    vite.config.ts                      # Vite bundler configuration
'''
```

---

## 4. API Reference Specifications

All API endpoints are mounted under the base URL ‘/api/v1‘ and accept/return JSON pay
loads.

### 4.1 Authentication (‘/api/v1/auth‘)

| Method | Endpoint | Description | Auth Required |
| :--- | :--- | :--- | :--- |
| ‘POST‘ | ‘/api/v1/auth/login‘ | Authenticate with work email & password | None |
| ‘POST‘ | ‘/api/v1/auth/demo-login‘ | 1-click persona login (‘role‘ or ‘email‘) | No
ne |
| ‘GET‘ | ‘/api/v1/auth/me‘ | Fetch active user credentials and employee profile | Be
arer Token |
| ‘POST‘ | ‘/api/v1/auth/logout‘ | Invalidate active user session | Bearer Token |

### 4.2 Workforce & Employees (‘/api/v1/employees‘)

| Method | Endpoint | Description | Min Role |
| :--- | :--- | :--- | :--- |
| ‘GET‘ | ‘/api/v1/employees‘ | List all employees (supports ‘?search=‘ and ‘?departm
ent=‘) | Employee |
| ‘GET‘ | ‘/api/v1/employees/:id‘ | Get detailed employee record by ID | Employee |
| ‘POST‘ | ‘/api/v1/employees‘ | Onboard a new employee | HR |
| ‘PUT‘ | ‘/api/v1/employees/:id‘ | Update profile, role, designation or department |
HR |
| ‘DELETE‘ | ‘/api/v1/employees/:id‘ | Deactivate/terminate employee record | HR |

### 4.3 Time & Attendance (‘/api/v1/attendance‘)

| Method | Endpoint | Description | Min Role |
| :--- | :--- | :--- | :--- |

| ‘GET‘ | ‘/api/v1/attendance/me‘ | Get logged-in user’s today status & personal history | Employee |
| ‘POST‘ | ‘/api/v1/attendance/clock-in‘ | Punch-in for today’s shift (detects late arrivals) | Employee |
| ‘POST‘ | ‘/api/v1/attendance/clock-out‘ | Punch-out and calculate total worked duration | Employee |
| ‘GET‘ | ‘/api/v1/attendance‘ | View organizational attendance records | Manager / HR |
| ‘POST‘ | ‘/api/v1/attendance/:id/correct‘ | Manual adjustment with mandatory reason | HR / Manager |

### 4.4 Leaves & Time Off (‘/api/v1/leaves‘)
| Method | Endpoint | Description | Min Role |
| :--- | :--- | :--- | :--- |
| ‘GET‘ | ‘/api/v1/leaves/balances‘ | Fetch personal leave balance quotas | Employee |
| ‘GET‘ | ‘/api/v1/leaves‘ | View requests (filtered by team for managers, all for HR) | Employee |
| ‘POST‘ | ‘/api/v1/leaves‘ | Apply for leave (validates balance availability) | Employee |
| ‘POST‘ | ‘/api/v1/leaves/:id/cancel‘ | Cancel an unapproved pending request | Employee |
| ‘POST‘ | ‘/api/v1/leaves/:id/manager-decision‘ | Stage 1 approval/rejection decision | Manager |
| ‘POST‘ | ‘/api/v1/leaves/:id/hr-decision‘ | Stage 2 final approval/rejection decision | HR |

### 4.5 Operational Modules
- **Departments (‘/api/v1/departments‘)**: Full CRUD and budget tracking.
- **Announcements (‘/api/v1/announcements‘)**: Company-wide broadcasts with priority levels.
- **Trainings (‘/api/v1/trainings‘)**: Workshop catalog, enrollment, and completion progress.
- **Reports (‘/api/v1/reports‘)**: Aggregated metrics and authorized CSV exports.
- **Audit Logs (‘/api/v1/audit-logs‘)**: Immutable compliance log history.

---

## 5. Role-Based Permissions Matrix

| Feature / Action | Employee | Manager | HR Admin |
| :--- | :---: | :---: | :---: |

| **Punch In / Clock Out** | Yes | Yes | Yes |
| **View Personal Timesheets** | Yes | Yes | Yes |
| **Apply for Leave** | Yes | Yes | Yes |
| **Cancel Own Pending Leave** | Yes | Yes | Yes |
| **Review Team Leave Requests (Stage 1)** | No | Yes | Yes |
| **Final Leave Approval (Stage 2)** | No | No | Yes |
| **Manual Attendance Corrections** | No | Yes (Team) | Yes (Org) |
| **Add / Edit Employee Records** | No | No | Yes |
| **View Employee Compensation** | No | No | Yes |
| **Create Department Units & Budgets** | No | No | Yes |
| **Publish Company Announcements** | No | No | Yes |
| **View Audit Trail Logs** | No | No | Yes |
| **Export Sensitive CSV Reports** | No | Yes (Team) | Yes (Full) |

---

## 6. Development & Operational Commands

‘‘‘bash
# Start development server on port 3000
npm run dev

# Run automated 17-point quality assurance test suite
npm test

# Run TypeScript compiler type-check
npm run lint

# Create production build bundle
npm run build

# Start production server
npm start
‘‘‘

---

## 7. Security Architecture
- **Cryptographic Hashing**: All seeded and user passwords are encrypted with ‘bcrypt
  js‘ salt rounds.
- **JWT Authentication**: Bearer tokens are signed with cryptographic secrets and val
  idated on every protected API endpoint.

- **No Client Secrets Exposed**: Server configuration and connection strings are isolated to backend Node processes.
- **Audit Accountability**: Every privileged administrative operation writes an immutable record containing timestamp, IP address, user identity, and payload changes.